

PDF AI Reader 大作业报告

一、程序功能介绍

1.1 项目定位

PDF AI Reader 是一款面向学术论文阅读场景的桌面端智能 PDF 阅读器，基于 Python 3.14 和 PySide6 构建。它的核心设计理念是：把论文从一份静态的 PDF 文件，变成一个可以边读、边翻译、边提问、边追溯证据的交互式工作台。

与市面上常见的翻译插件或网页版阅读工具不同，PDF AI Reader 追求的是深度阅读体验——它不仅提供翻译和问答，还尝试在原文段落间做就地交互，避免用户在多个窗口之间来回跳转而打断阅读思路。

1.2 核心功能模块

(1) PDF 阅读与渲染

应用的基础能力是 PDF 文件的打开、渲染、缩放和导航。技术上采用 PyMuPDF 作为 PDF 解析后端，UI 层实现了基于瓦片（tile）的渐进式渲染方案。页面被切分为 256x256 像素的瓦片单元，根据当前视口位置按需请求渲染，配合 LRU 内存缓存和页面级 pixmap 缓存，在内存占用和滚动流畅度之间取得平衡。

阅读器支持以下导航方式：鼠标滚轮/触控板滚动、Ctrl+= 和 Ctrl+- 缩放、工具栏页码跳转、左侧目录面板点击跳转。目录结构从 PDF 原生大纲中提取，对于缺少大纲的文档，会自动根据段落字号和加粗特征生成结构化的章节目录。

(2) 段落交互与裂缝式翻译

这是 PDF AI Reader 最具创意的交互设计。用户在 PDF 页面上任意段落上双击，该段落下方会像纸张裂开一样展开一个交互区域（项目内部称其为 Split Widget），显示该段落的翻译结果。再次双击则折叠收起，恢复正常的页面布局。

这个设计解决了传统侧边栏翻译的一个根本问题：用户的目光需要反复在原文区域和翻译区域之间跳转，容易丢失语境。而裂缝式交互让翻译直接出现在原文正下方，阅读顺序自然且连贯。

翻译链路内部做了多项工程优化：公式占位保护（将 LaTeX 公式在翻译前替换为占位符，翻译后再恢复）、术语表注入（将用户定义的领域术语映射注入系统提示词）、AI 结果缓存（相同内容不重复请求）、以及流式输出（翻译结果逐字呈现，减少等待焦虑）。

(3) 全文问答与证据追溯

右侧面板提供了一个全文问答入口。用户可以针对当前打开的论文提出任意问题，系统会先从本地知识库中检索相关段落，然后将检索到的证据连同问题一起发送给大语言模型生成回答。

知识库检索是问答质量的关键。项目支持两种后端：基于 ChromaDB 的向量检索和基于 SQLite FTS5 的全文检索，可以在配置文件中切换。检索结果经过混合重排序（向量相似度 + 词汇覆盖率 + bigram 匹配），最终作为证据提供给模型。回答中会标注证据来源，用户可以双击证据项跳转到原文对应位置进行核实。

(4) 公式识别与处理

公式处理是学术论文阅读中最困难的部分之一，也是本项目投入了大量工程精力的模块。项目设计了一套多轮异步公式索引流水线，从 PDF 文件的多种事实来源中逐步提取和验证公式内容：

- **r0 轮**：从 born-digital PDF（即由 LaTeX/Word 等排版软件直接生成的 PDF，而非扫描件）的文本层、glyph 坐标、字体信息和向量图形中提取结构事实。这类 PDF 在生成时保留了字符的精确位置和字体信息，因此可以反向推断公式的结构。
- **r0.5 轮**：符号身份修复。对 PDF 中无法直接映射到 Unicode 的 glyph（如 Type3 字体、CID 字体），通过 glyph 名称表、同字体锚点等线索推断其真实的数学符号身份。
- **r1 轮**：从本地缓存中查找已有的 OCR 识别结果。
- **r2 轮**：使用本地 OCR 模型（基于 Pix2Text）对公式区域进行高精度识别。
- **r2a 轮**：TinyBDMath 结构模型候选。这是项目自研的 born-digital 公式结构恢复系统，从 PDF glyph 级别的坐标和字体事实出发，经过图解析器、约束解码和 LaTeX 解码器，生成结构化的 LaTeX 候选。这是整个公式处理链路中最具技术含量的部分。
- **r3 轮**：云端大模型语义复核。将多个候选公式发送给云端模型进行语义层面的审查和融合。
- **r4 轮**：知识图谱更新。将审核通过的公式纳入文档级的知识图谱。
- **r5 轮**：增量写回知识库。只有被接受（accepted）的公式才会最终写入知识库，避免低质量结果污染检索质量。

整个流水线是异步后台运行的，不会阻塞用户的正常阅读。公式识别的进度和结果可以在界面中查看。

(5) 术语表管理

针对学术翻译中常见的术语一致性问题，项目提供了术语表管理功能。用户可以按学科领域（如计算机/机器学习、数学、物理）维护中英文术语对照表，支持强制翻译、别名、备注等字段。翻译时，术语表会自动注入到系统提示词中，确保关键术语的翻译与用户的偏好一致。

(6) 段落批注

用户可以在任意段落的右键菜单中选择批注/备注，在裂缝区域中录入个人的阅读笔记。批注内容按文档和段落 ID 持久化到本地 JSON 文件，不会上传到任何云端服务。同一段落可以同时有翻译和批注，阅读顺序为原文 → 翻译 → 备注。

(7) 多标签阅读会话

主窗口顶部提供类似浏览器的标签条，用户可以同时打开多个 PDF 文档，每个标签独立保存阅读位置（页码和滚动偏移量）。关闭应用后，完整的标签会话会持久化到本地 JSON 文件；下次启动时自动恢复所有标签及其阅读位置。

(8) 主题切换

提供四套配色主题：深色（黑底白字）、米黄（暖色护眼）、浅灰、黑白。主题通过 Qt 的 QPalette 机制全局生效，翻译框和批注框保持独立的配色不受主题影响。

二、项目架构与模块设计

2.1 整体架构

项目采用分层架构设计，从上到下分为四层：

UI 层 (src/ui/)	主窗口、PDF阅读器、裂缝组件、术语表对话框
应用流程层 (src/app/)	翻译流程、问答流程、公式索引流程、图谱流程
核心业务层 (src/core/)	PDF引擎、AI引擎、知识库引擎、公式处理
基础设施层 (src/infra/) + 数据访问层 (src/data/)	缓存、配置、向量存储、持久化

各层之间的依赖方向严格向下：UI 层调用应用流程层，应用流程层调用核心业务层，核心业务层调用基础设施和数据层。层与层之间通过 Qt 的信号/槽机制实现异步通信，避免阻塞 UI 线程。

2.2 各层模块详解

2.2.1 基础设施层 (src/infra/)

这一层提供最底层的通用能力，不涉及任何业务逻辑。

file_hash.py：文件哈希计算工具。使用 SHA-256 算法对文件内容进行分块哈希，生成文档的唯一标识符。这个哈希值在整个系统中被广泛使用——知识库的命名空间、缓存的键、会话恢复的依据都依赖它。

tile_cache.py：瓦片内存缓存。基于 Python 的 OrderedDict 实现 LRU（最近最少使用）淘汰策略，默认容量为 200MB。缓存的键由页码、瓦片坐标和缩放级别组成，即同一页面在不同缩放级别下的瓦片是独立

缓存的。

tile_renderer.py：瓦片渲染调度器。这是 PDF 渲染性能的核心组件。它将每个页面切分为 256x256 像素的瓦片，使用 QThreadPool（2 个工作线程）并发渲染。调度器实现了优先级队列：视口中心附近的瓦片优先渲染，边缘瓦片延后。同时采用代数（generation）机制处理请求取消——当用户快速滚动时，旧的渲染请求会被标记为过期，其返回结果会被丢弃，避免渲染错误的位置。

page_cache.py：页面级 LRU 像素图缓存。与瓦片缓存互补，当页面较小时直接缓存整页的渲染结果。支持从高分辨率缓存缩放到低分辨率请求，避免重复渲染。

ai_cache.py：AI 结果持久化缓存。基于 SQLite 数据库，缓存翻译、OCR、摘要和问答的结果。缓存的键由文档哈希、段落 ID、结果类型和内容哈希组成。使用 WAL（Write-Ahead Logging）日志模式支持并发读写。

2.2.2 数据访问层（src/data/）

config_manager.py：配置管理器。负责 YAML 配置文件的读写，并通过 Pydantic 模型进行数据校验。支持部分更新（deep merge）和热加载。API Key 的查找支持多来源：优先从配置文件读取，其次从环境变量读取。

chroma_repo.py：ChromaDB 向量存储封装。管理 ChromaDB 集合的创建、增删改查。集合名称使用 PDF 文档的哈希值作为命名空间，确保不同文档的知识库相互隔离。

annotation_store.py：批注持久化存储。使用 JSON 文件配合 QLockFile 实现跨进程安全的数据写入。写入操作采用先写临时文件再原子替换的策略，防止写入过程中断导致数据损坏。

glossary_repo.py：术语表持久化。按学科领域分别存储为 JSON 文件，支持 JSON 和 CSV 两种格式的导入。

2.2.3 核心业务层（src/core/）

这是整个项目最重要的代码层，承载了所有业务逻辑。

models.py —— 数据模型定义

使用 Pydantic 定义了系统中所有核心数据结构：

- **DocumentBlock**：文档段落的基本单元，包含段落 ID、页码、类型（正文/公式/标题/图片/表格）、文本内容、包围框坐标、所属章节标题等字段。这是系统中流转最频繁的数据结构，翻译、检索、问答、批注都围绕它展开。
- **AppConfig**：应用配置的顶层模型，聚合了模型配置（本地/云端/嵌入模型名称）、RAG 配置（检索后端类型、候选池大小、混合重排序开关）、路由策略（各任务优先使用本地还是云端模型）和 UI 配置（主题、字体）。
- **BlockType 枚举**：PARAGRAPH、FORMULA、HEADING、IMAGE、TABLE 五种段落类型。
- **SplitMode 枚举**：TRANSLATION、QUESTION、EXPLANATION、ANNOTATION 四种裂缝交互模式。

pdf_engine.py —— PDF 文档引擎

DocumentEngine 是 PDF 处理的总入口，协调文档的打开、解析、渲染全流程。核心设计决策包括：

- **两阶段解析**：打开文档时先解析前 8 页以快速展示首屏内容，剩余页面在后台线程中继续解析。解析完成后触发 Pix2Text 的公式检测（MFD）进行公式区域细化。
- **DocumentChunker**：段落分块器，是整个解析链路中最复杂的组件之一。它调用 PyMuPDF 的 `get_text("dict")` 获取页面级别的文本块信息，然后依次执行：按字体和 Unicode 范围判断文本块类型 → 识别双栏布局并交错排列 → 检测标题（基于字号和加粗） → 检测公式（基于数学字体名如 CM/STIX、Unicode 数学符号区间、LaTeX 命令模式） → 合并 born-digital 公式事实 → 标记被公式覆盖的阴影区域避免重复。
- 异步页面渲染通过 `_PageRenderTask`（`QRunnable`）在线程池中执行，渲染完成后通过信号通知 UI。

ai_engine.py —— AI 引擎

AIEngine 是调用大语言模型的总入口，管理翻译、问答和嵌入三种 AI 能力。

- **BaseLLMClient 抽象基类**：定义了 `generate`、`generate_stream`、`embed_batch` 等统一接口。
- **LiteLLMClient**：封装 LiteLLM 库，统一调用各种云端模型（DeepSeek、GPT-4o 等），内置 2 次重试机制。
- **HashingEmbeddingClient**：当没有可用的嵌入模型时，使用 BLAKE2b 哈希算法生成确定性嵌入向量作为降级方案。这是一个务实的设计——即使没有 GPU 或嵌入服务，知识库检索仍能以有限的质量运行。
- **HybridModelRouter**：混合路由策略，根据配置的 `RoutingStrategy`（本地优先/仅云端/仅本地）和当前模型可用性动态选择调用哪个后端。问答和摘要任务还支持使用独立的推理模型（`reasoning_client`）。
- **TranslationService**：翻译服务，核心特征是公式保护（将 LaTeX 公式替换为占位符 → 翻译 → 恢复公式）、术语表注入（将用户的术语映射格式化为提示词前缀）和领域特化提示词（针对定理、证明等数学文本的特殊处理）。
- **QAService**：问答服务，构建包含当前段落、检索证据和对话历史的完整上下文，支持证据引用标注和追问建议生成。
- 翻译和问答的执行在独立的 `QThread` 工作线程中进行，通过信号将流式结果逐 token 发送回 UI 线程。

knowledge_engine.py —— 知识库引擎

KnowledgeEngine 负责知识库的构建和检索。构建流程为：遍历文档所有段落 → 批量嵌入 → 存入后端。检索流程为：嵌入查询文本 → 从后端召回候选 → 混合重排序 → 返回最终结果。

混合重排序是检索质量的提升手段。单纯依赖向量相似度有时会召回语义相关但不包含关键术语的段落。混合重排序在向量分数（权重 0.55）的基础上叠加词汇覆盖率分数和 bigram 匹配分数（合计权重 0.45），使同时满足语义相关和词汇匹配的段落排名更靠前。

knowledge_backends.py —— 知识库后端抽象

定义了 KnowledgeIndexBackend 抽象接口，提供了三种实现：

- **LegacyChromaBackend**：直接操作 ChromaDB 的集合，批量 upsert（每批 512 个段落）。
- **LlamaIndexChromaBackend**：通过 LlamaIndex 框架操作 ChromaDB，集合名带有版本前缀以避免不同版本的索引冲突。
- **SQLiteFtsBackend**：基于 SQLite FTS5 全文搜索引擎，零额外依赖。使用 BM25 算法进行相关性评分。对于不需要语义理解的精确关键词搜索场景，FTS5 的效果往往优于向量检索。

navigator.py —— 导航管理器

管理文档的目录（TOC）和书签。目录的生成包含多项防御性处理：检测和修复乱码（mojibake）、过滤公式片段误入目录、识别中文数字编号等。书签支持用户手动添加和 AI 自动建议（利用定理、引理、定义等结构化块自动推荐值得标记的位置）。

glossary_manager.py —— 术语表管理器

管理多学科术语表，核心功能是在翻译时将术语映射格式化为提示词注入。冲突解决策略为：当同一个英文术语在不同学科中有不同翻译时，优先使用用户设定的首选学科的解释。

formula_detector.py —— 公式检测器

公式检测器的抽象接口，当前主要实现是 Pix2TextMFDDetector，利用 Pix2Text 库的 MathFormulaDetector（基于深度学习的数学公式检测模型）来识别页面中的公式区域。检测流程包括：候选页面排序（优先检测公式密度高的页面）、去重（基于包围框的 IoU 重叠率，阈值 85%）、以及预算限制（每次调用限制 OCR 数量）。

math_ocr.py —— 公式 OCR 服务

MathOCR 是一个进程级单例，封装了公式图片到 LaTeX 的识别能力。特点包括：持久化 SQLite 缓存（按模型和图片哈希去重）、批量处理（每批 4 张图片）、以及 CPU 亲和性设置（限制 OpenMP 线程数以控制资源占用）。

TinyBDMath 公式结构恢复系统（9 个文件）

这是项目中最具原创性的技术模块。与传统的将公式当作图片做 OCR 不同，TinyBDMath 的思路是利用 born-digital PDF 中保留的精确 glyph 坐标和字体元数据，将公式的结构恢复问题建模为一个结构预测问题。

核心流水线如下：

1. **特征提取 (tinybdmath_features.py)**：从 PDF glyph 级别的坐标、字体、大小信息中提取两两 glyph 之间的关系特征。这些特征包括水平距离、垂直偏移、大小比率、基线对齐程度等数值特征。
2. **图解析器 (tinybdmath_graph_parser.py)**：使用一个纯 Python 实现的前馈神经网络（无需 PyTorch 运行时，模型参数以 JSON 格式导出），对 glyph 之间的关系进行分类——水平并列、上标、下标、分式分子、分式分母、根号内部等。
3. **约束解码 (tinybdmath_constrained_decode.py)**：在图解析器的原始输出上施加结构约束（如一棵树中不能有环、一个节点不能同时是上标和下标等），通过 beam search 生成符合语法的最优结构。

4. **CSLT 目标树 (tinybdmath_cslt_schema.py + tinybdmath_target_tree.py)**: 定义了规范符号布局树 (Canonical Symbol Layout Tree, CSLT) 的数据结构, 作为结构预测的目标格式。从 KaTeX 的解析树自动构建 CSLT, 用于训练数据的生成。
5. **LaTeX 解码器 (tinybdmath_latex_decoder.py)**: 将 CSLT 结构树转换为 LaTeX 字符串。
6. **布局验证器 (tinybdmath_layout_verifier.py)**: 将解码后的 LaTeX 渲染结果与 PDF 原文的视觉布局进行对照验证。
7. **对齐模块 (tinybdmath_alignment.py)**: 将 PDF glyph 图与 CSLT 目标树进行节点级别的对齐, 为图解析器生成训练标签。
8. **符号等价性 (tinybdmath_symbol_equivalence.py)**: 处理 Unicode 符号的各种规范化形式之间的等价判断。

公式相关辅助模块

- **pdf_glyph_graph.py**: 从 PyMuPDF 的页面数据中提取原始的 glyph 图结构, 包含 glyph 节点、矢量图形区域、阅读顺序边等。
- **born_digital_math.py**: 从 PDF 事实中提取数学证据, 包括公式区域的检测、行间公式的分割、分式线条的识别等。
- **born_digital_formula_extractor.py**: 整合 glyph 图和符号修复, 生成 r0 轮的 born-digital 公式候选。
- **symbol_identity_repair.py**: 处理 PDF 中无法直接识别身份的 glyph (Type3 字体、CID 字体等), 通过 AGL (Adobe Glyph List) 名称表、TeX glyph 名称表等映射关系进行推断。
- **latex_macro_expander.py**: LaTeX 宏展开器, 用于训练数据预处理。
- **latex_math_source_parser.py**: 从 LaTeX 源码中提取数学公式区间。
- **latex_mathml_extractor.py**: 通过 KaTeX 将 LaTeX 转换为 MathML 或解析树, 用于训练数据准备。

service_container.py —— 依赖注入容器

这是连接所有模块的粘合剂。ServiceContainer 支持三种生命周期:

- **Instance (实例)**: 预先构造好的对象直接注册, 每次获取返回同一个。
- **Singleton (单例)**: 注册构造工厂函数, 首次获取时创建并缓存, 后续返回缓存实例。
- **Factory (工厂)**: 每次获取都调用工厂函数创建新实例 (例如 DocumentEngine, 因为每个打开的文档需要独立的引擎实例)。

容器还管理各组件的关闭顺序——shutdown 时按注册的逆序调用每个 singleton 的 close/shutdown 方法, 确保资源正确释放。

error_handler.py —— 错误处理

集中化的错误处理机制。定义了四级严重程度 (INFO/WARNING/ERROR/CRITICAL), 支持按错误类型注册回调函数, 对于严重错误会弹出 QMessageBox 对话框。同时安装了全局的 sys.excepthook, 确保未捕获的异常也能被记录。

2.2.4 应用流程层 (src/app/)

这一层位于核心业务层和 UI 层之间, 负责编排跨多个核心服务的业务流程。

document_flow.py: 文档生命周期协调器。管理 PDF 打开 → 解析 → 知识库构建的完整流程。在首屏页面就绪后立即触发知识库构建，后续页面解析完成后增量更新。

translate_flow.py: 翻译流程。在 AI 引擎翻译之前先检查 AI 缓存，命中则直接返回；未命中则委托 AI 引擎执行翻译，完成后写回缓存。还包含短路判断逻辑：纯公式文本、过短文本、中文文本直接返回原文，避免浪费 API 调用。

explain_flow.py: 概念解释流程。对公式块触发按需 OCR 识别，然后构建解释类问题。

ask_flow.py: 全文问答流程。从 KnowledgeEngine 检索相关证据，无证据时降级为全文均匀采样，然后构建完整问答上下文发送给 AI 引擎。

formula_index_flow.py: 公式索引总流程。管理后台多轮公式 OCR 的排队、调度和结果持久化。支持两种扫描模式：块级扫描（针对已检测到的公式区域）和页面级扫描（针对整页的 MFD 检测 + born-digital 结构提取）。

formula_index_scheduler.py: 公式扫描调度策略。根据触发类型分配不同的扫描预算：视口触发 2 页、证据触发 4 页、用户手动触发 6 页、后台空闲 8 页。

formula_index_store.py: 公式索引的持久化存储。基于 SQLite，管理公式索引任务的全生命周期（排队 → 运行中 → 完成/失败/跳过），以及多轮识别结果、审核决策和融合记录的存储。

formula_semantic_review.py: 公式语义审核服务（r3 轮）。将所有候选公式连同融合证据构建为提示词，发送给云端大模型进行语义审查。

formula_acceptance_review.py: 公式接收审核服务。提供接受/拒绝/修订公式候选的 API，所有操作都有审计追踪记录。

formula_knowledge_graph.py: 公式知识图谱服务（r4 轮）。将审核通过的公式纳入文档级结构图谱。

formula_knowledge_update.py: 公式知识更新服务（r5 轮）。将接受的公式增量写回 KnowledgeEngine。

graph_index_flow.py / graph_index_store.py: 通用知识图谱索引的流程控制和持久化，从文档结构中提取页面 → 段落 → 公式 → 定理的关系图。

tinybdmath_candidate_service.py: TinyBDMath 结构候选服务（r2a 轮）。消费 r0/r0.5 的结构证据，运行完整的 TinyBDMath 流水线（特征提取 → 图解析 → 约束解码 → LaTeX 解码），生成结构化的公式候选。

test_command_bridge.py: 测试命令桥接。仅在 --test-mode 下激活，通过文件轮询接收测试命令（如跳转到某页、打开翻译等），并发送事件到输出文件，供自动化测试框架使用。

2.2.5 UI 层 (src/ui/)

main_window.py: 主窗口，项目中最庞大的单个文件（约 4400 行）。管理菜单栏、工具栏、左侧目录面板、中央 PDF 阅读器、右侧 AI 工具集面板和状态栏的布局与交互。关键职责包括：多标签管理

(QTabBar 的创建、切换、拖拽排序和会话持久化)、面板的显示/隐藏/浮动、主题切换的全局协调、以及各流程协调器(翻译流程、问答流程等)的初始化和连接。

pdf_viewer.py: PDF 阅读器组件, 基于 QScrollArea 实现。核心设计包括:

- **_VirtualPageLayout**: 纯 Python 的虚拟页面布局计算器, 不依赖 Qt widget 树。通过二分查找实现 $O(\log n)$ 的坐标→页码映射。这使得滚动时的页面定位非常高效。
- **_LazyPageWidget**: 懒加载的页面组件。只有视口附近(默认上下各约 15 个页面的范围)的页面才有活跃的 widget, 之外的页面 widget 被回收。这种虚拟化策略使得打开数百页的 PDF 也不会创建大量 widget。
- **PdfViewer**: 整合 TileRenderer、TileCache 和 PageCache, 管理视口内的瓦片请求、页面预加载和内存释放。支持方向感知的预加载策略——根据用户最近的滚动方向预测接下来可能进入视口的页面并提前渲染。

split_widget.py: 裂缝式交互容器。这是项目 UI 设计中最有特色的组件。核心机制是: 在 PDF 页面上选中段落对应的包围框后, 利用 QPropertyAnimation 在段落下方展开一个区域。展开的动画使用 OutCubic 缓动曲线(250ms), 收起使用 InCubic 曲线(200ms)。内容通过 QWebEngineView 渲染, 支持 Markdown 和 KaTeX 公式排版。WebView 使用对象池模式(WebViewPool)维护一个预热好的备用 WebView, 减少展开时的等待时间。收起时 WebView 会被冻结并截图, 以便下次展开时先显示截图作为占位。

paragraph_widget.py: 段落覆盖层。透明背景的 QWidget 子类, 叠加在 PDF 页面上方, 根据 DocumentBlock 的包围框坐标定位。鼠标悬停时高亮(正文紫色、公式灰色), 定理类段落显示左侧橙色边框。支持双击触发翻译、右键菜单触发批注或解释。

theme.py: 主题系统。通过 QPalette 和局部 QSS (Qt 样式表) 实现四套主题。与全局 QSS 不同, 本地 QSS 只作用于特定 widget, 避免样式冲突。

glossary_dialog.py: 术语表管理对话框。左侧领域列表 + 右侧术语表格的双栏布局, 支持搜索过滤、增删改查、JSON/CSV 导入。

三、小组成员分工

王昱焜(组长): 完成项目的设计构思、功能设计, 进行了主体框架的搭建, 编写了绝大部分代码, 完成了裂缝翻译、AI问答、公式识别小模型训练等核心功能的构想、设计和实现。

梁灏: 完善功能和底层逻辑, 增加批注框功能, 协同打磨多主题, 优化目录提取和打开方式, 实现多窗口标签化等页面优化。

李林歌: 主要完善了呈现形式, 做了外观上的改善, 制作演示视频, 更正了部分情况出现错误弹窗的bug。

四、总结与反思

4.1 项目亮点

工程架构的成熟度。项目采用了清晰的分层架构，各层职责明确、依赖方向严格。依赖注入容器的设计让各模块之间的耦合度保持在较低水平，方便测试和替换实现。信号/槽机制的使用使得异步操作（AI 调用、PDF 渲染、公式识别）不会阻塞 UI 线程。

公式处理链路的深度。TinyBDMath 系统是项目的技术核心，体现了从 PDF 底层事实出发进行结构化公式恢复的完整思路。与传统 OCR 方案不同，这条技术路线不需要训练大型视觉模型，而是利用 born-digital PDF 中已有的精确坐标信息，将问题转化为结构预测。多轮异步流水线的设计也保证了公式处理不会影响正常的阅读体验。

用户体验的细节打磨。裂缝式交互避免了眼动跳转、多标签会话持久化让阅读可以跨启动继续、AI 缓存避免了重复请求的等待和费用、瓦片化渲染保证了长文档的滚动流畅度。这些细节单独看都不是惊天动地的创新，但组合在一起构成了一个可以日常使用的实用工具。

4.2 技术选型反思

PySide6 的选择是合理的。Qt 框架在桌面端跨平台开发中有着成熟的生态和丰富的组件，对于 PDF 阅读器这种需要复杂渲染和交互的应用，Qt 的 Graphics View 框架和 WebEngine 集成提供了必要的能力。

PyMuPDF 的适配工作量较大。PyMuPDF 虽然是优秀的 PDF 库，但其文本提取 API 的行为在不同 PDF 生成器之间存在差异。项目中有大量代码用于处理字体名称规范化、mojibake 检测、Column 检测等边界情况，这些本质上是在弥补 PDF 格式本身的不规范性。

纯 Python 推理的 TinyBDMath 是有趣的工程选择。不使用 PyTorch 运行时，而是将模型参数导出为 JSON 并通过纯 Python 前馈网络进行推理，这种做法虽然推理性能不如 GPU 方案，但消除了重量级依赖，简化了部署。对于 born-digital PDF 公式恢复这个特定场景（单篇论文的公式数量通常不超过几百个），纯 Python 推理的性能是可接受的。

4.3 对未来开发的建议

公式恢复应继续作为主线。这是项目区别于通用 PDF 阅读器 + 翻译插件组合的核心竞争力。当前的 TinyBDMath 路线在技术上是独特的，值得继续投入。

知识图谱功能可以进一步发掘。目前文档级的结构图谱（页面→段落→公式→定理）已经建立，但如何利用这些结构化信息来增强问答和推荐体验，还有很大的想象空间。

测试覆盖需要向端到端方向倾斜。项目有大量单元级和模块级测试（总计数百个测试用例），但桌面端 E2E 测试的覆盖相对薄弱。公式还原质量的验证尤其需要系统化的端到端测试。

日期：2026 年 7 月 6 日